

Some Advanced Topics in Shared Memory Systems

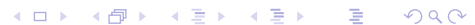
Computer Systems Group

Research School of Computer Science
Australian National University
Canberra, Australia

May 22, 2016



ANU College of
Engineering & Computer Science



Lock Free Synchronization

Outline



1 Lock Free Synchronization

2 Transactional Memory





'Lock-free' Data Structures: Motivation

- consider the **atomic test-and-set** operation

```
atomic int testAndSet(volatile int *Lock) {int lv = *Lock; *Lock = 1; return lv;}
```

- synchronizes *whole* memory system (down to Last Level Cache (LLC)), degrades memory access progress for all
- mutual exclusion via test-and-set can be modelled by:

```
volatile int lock = 0; //0..1; 1=locked, 0=unlocked
```

```
1 // thread 0
2 while (1) {
3     // non-critical section 0
4     ...
5     while (testAndSet(&lock) != 0)
6         { /*spin*/ }
7
8     // critical section 0
9     ...
10    lock = 0;
11 }
```

```
1 // thread 1
2 while (1) {
3     // non-critical section 1
4     ...
5     while (testAndSet(&lock) != 0)
6         { /*spin*/ }
7
8     // critical section 1
9     ...
10    lock = 0;
11 }
```

- with p threads under heavy contention (time in non-critical becomes small), there will be $O(p^2)$ atomics!! Can we do better?

Navigation icons: back, forward, search, etc.



Improving Locked Access to Data Structures

- only try atomics when we see lock is free: reduces #atomics to $O(p)$

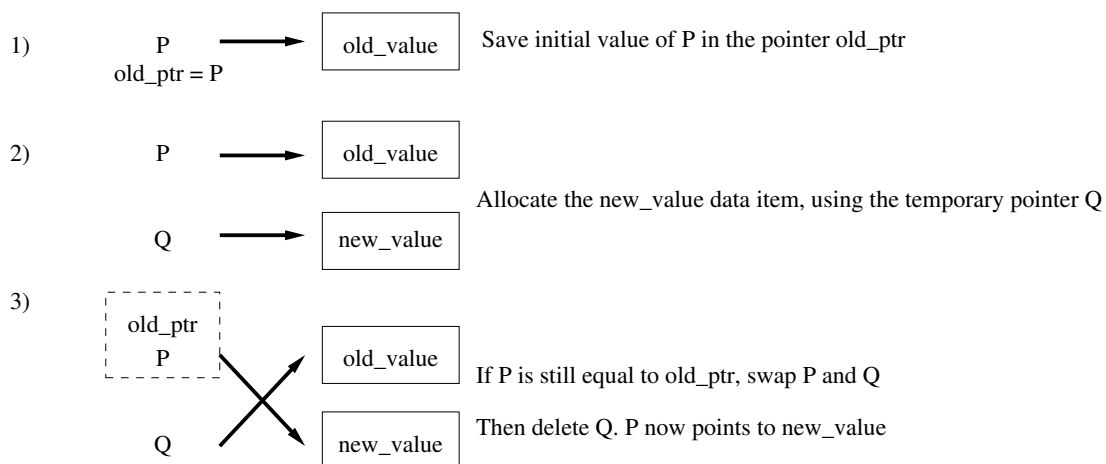
```
1 do {
2     while (lock == 1) { /*spin*/ }
3 } while (testAndSet(&lock) != 0);
```

- use **adaptive spin-locks**: if lock is not acquired after a certain time, the thread yields
 - Pthreads mutexes should do this; especially useful if $p > N$
- use algorithms requiring a (small) constant number of atomic operations per access of the critical region
 - can be a general mutex algorithm, or data structure-specific
- better still (possibly), use algorithms using no atomic operations
 - Lamport's Bakery Algorithm: still has $O(p^2)$ costs, assumes order of updates made by one thread is seen by other threads

Navigation icons: back, forward, search, etc.

- each pair of threads points to a leaf node in a tree
 - each node has a `ctr` (init. to 2), and a `sense` flag (init. to 0)
- algorithm: begins on all leaf nodes (each thread has `threadSense=1` flag):
 - `if (fetchAndDecrement(ctr) == 1)`
 call algorithm on parent node
 `ctr = 2, sense = !sense`
 wait until `threadSense == sense`
 - then, as leaving the barrier, `threadSense = !threadSense`
- notes:
 - last thread to reach each node continues up the tree
 - the thread that reaches root begins 'wakeup' (reversing `sense`)
 - upon wakeup, a thread releases siblings at each node along path
- performance: $2\times$ the atomic operations, but can distribute memory locations of node data (e.g. across different LLC banks)
 - atomics can be avoided by the **scalable tree barrier**! (simply replace `ctr` with 2 adjacent byte flags, one for each thread)
- note: an 8-byte word can be used for atomic-less barrier ($p \leq 8$)

- Mutual exclusion: pessimistic approach, plan for worst. “A race condition may happen here, so we will make sure that it never does”
- Lock-Free: optimistic, plan for the best. “We go ahead, and if a race condition occurs we start all over again”



Compare and Swap (CAS) Atomic Operation



```

function cas(p : pointer to int, old : int, new : int) returns bool {
2   if *p != old {
4       return false
    }
    *p = new
6   return true
}

```

However problems can arise if there is any chance of the old value and the new value being the same but a change has actually taken place. This may occur if there is a wrapped version counter or a recycling of address. The so called ABA problem. The problem overcome using some form of double compare and swap (DCAS or CAS2).

'Lock-Free' Stack



```

1 class LFStack {
2     int top, nPop;
3     LFStack(int Nc) { N = Nc; nPop = top = 0; buf = new int [N]; }
4     bool push(int v) {
5         while (1) {
6             int oldTop = top;
7             if (oldTop == N) return false; //caller must try again
8             if (dcas(&top, oldTop, oldTop+1,
9                     &buf[oldTop], buf[oldTop], v))
10                return true; //oldTop remained a valid stack top
11        }
12    }
13    bool pop(int *v) {
14        while (1) {
15            int oldTop = top, oldNPop = nPop;
16            if (oldTop == 0) return false; //caller must try again
17            *v = buf[oldTop-1];
18            if (dcas(&top, oldTop, oldTop-1,
19                    &nPop, oldNPop, oldNPop+1))
20                return true; //oldTop remained a valid stack top
21        }
22    }
23 }; //note: dcas(&top, oldTop, oldTop-1, v, *v, buf[oldTop]) won't work!

```

dcas() is a 'double' compare-and-swap operation:

- in this case we use the op on the 1st word to ensure safety, and the 2nd to ensure data is added / removed atomically

Summary: Lock Free Synchronization



- Use fine-grained locking to reduce contention in operations on shared data structures
- Non-blocking solution to avoid overheads due to lock, but still requires appropriate memory fence operations
- Lock free design does not eliminate contention

Outline



1 Lock Free Synchronization

2 Transactional Memory

Transactional Memory - Motivations

or, problems with lock-based solutions:

- **convoying**: a thread holding a lock is de-scheduled, possibly holding up other threads waiting for a lock
- **priority inversion**: lower-priority thread is pre-empted while holding a lock needed by a higher priority thread
- **deadlock**: two threads attempt to acquire the same locks do so in different order
- inherently unsafe: relationship between data objects and locks is entirely by convention
- lacks the **composition** property: difficult to combine 2 concurrent operations using locks into a single larger operation
- pessimistic: in low-contention situations, their protection is only needed $< 1\%$ of the time!
 - recall overheads of acquiring a lock (even if free)

Navigation icons: back, forward, search, etc.

Transactions - Basic Ideas

- a **transaction** is a series of reads and writes (to global data) made by a single thread which execute **atomically**, i.e:
 - if it succeeds, no other thread sees any of the writes until it completes
 - if it fails, no change to (global) data can be observed
- it must also execute with **consistency**:
 - the thread does not see any (interfering) writes from other threads
- it has the following steps:
 - **begin**, read and writes to (transactional) variables, **end**
- if **consistency** is met, the **end** results in a **commit**; otherwise it **aborts**
- a condition that meets this is **serializability**:
 - if transactions cannot execute concurrently, **consistency** is assured
 - note: this assumes that (transactional) variables are only written (in some cases, read) by transactions

can also achieve this if no other thread accesses the affected variables during the transaction

Navigation icons: back, forward, search, etc.



Transactions - Programming Model

- e.g. to add an item `newItem` to a queue with tail pointer `tail`:

```

atomic {
2   struct node *newTail = malloc(sizeof(struct node));
   newTail->item = newItem;
4   newTail->next = tail;
   // tail must not be changed between these 2 points!
6   tail = newTail;
}

```

here, `tail` is the single read and write transactional variable

- the `atomic` block is executed atomically (it can be assumed that the underlying system will re-execute it until it can commit (what about the `malloc`??))
- e.g. code from **Unstructured Adaptive** benchmark of NAS suite

```

for (ije1 = 0; ije1 < nnje; ije1++) { ...
2   for (ije2 = 1; ije2 < nnje; ije2++)
       for (col = 1-shift; col < lx1-shift; col++) {
4       ig = idmo[v_end[ije2], col, ije1, ije2, iface, ie];
       #pragma omp atomic
6       tmor[ig] += temp[v_end[ije2], col, ije1] * 0.5;
       ... }
8   }

```

Navigation icons: back, forward, search, etc.



Limited Compare-and-Swap Implementation

- the **atomic compare-and-swap** operation may be expressed as:

```

typedef unsigned long uint64;
2   atomic uint64 cas(uint64 *x, uint64 x_old, uint64 x_new) {
       uint64 x_now = *x;
4       if (x_now == x_old) *x = x_new;
       return (x_now);
}

```

(1st 2 lines are implemented by a single `cas` instruction)

- we can implement the (essentially atomic part of) the code for queue insertion by:

```

do {
2   newTail->next = tail;
} while (cas(&tail, newTail->next, newTail) != newTail->next);

```

- and for the **Unstructured Adaptive** benchmark

```

1   do {
       x_old = tmor[ig];
3       x_new = temp[v_end[ije2], col, ije1] * 0.5 + x_old;
       } while (cas(&tmor[ig], x_old, x_new) != x_old);

```

Navigation icons: back, forward, search, etc.

Hardware TM: Basis on Coherency Protocols



- idea: tentative values can be held in cache lines (assuming each processor has a separate cache)
- review the standard **MESI** cache coherency protocol
 - cache line states:
 - Modified** only this cache holds the data; it is 'dirty', **Exclusive**: as before, but not 'dirty', **Shared**: > 1 caches hold data, not 'dirty', **Invalid**: this line holds no data
 - key transitions:
 - (a) **A**: load x (miss; then line in **E** state)
 - (b) **B**: load x (miss; then lines are in **S** state (**A** and **B**))
 - (c) **B**: store x (hit; then lines in **M** state (**B**) and **I** state (**A**))
 - (d) **A**: load x (miss; then **B** copies back line to memory and lines in **S** state (**A** and **B**))
 - when a line is *evicted* (due to a conflict with a new address needing to be loaded), the data is copied-back to memory

Hardware Transactional Memory (Bounded)



- add a **transactional bit** (**T**) for the state of each cache line
 - $T=1$ if entry is placed in cache on behalf of a transaction (o.w. 0)
- simply extend the **MESI** protocol as follows:
 - if a line with $T=1$ is invalidated or evicted, abort (with no copy-back)
 - note: can record fact that transaction will later abort instead
- abort causes lines with $T=1$ to be invalidated (no copy-back, if dirty)
- a commit requires all dirty lines with $T=1$ to be written **atomically** to memory (**T** bit is cleared)
- requires transactional variants to load and store instructions, plus an (attempt to) commit instruction (which clears all **T** bits)
- why this works:
 - if a line with $T=1$ is invalidated, we have a R-W or W-W conflict
 - if a line with $T=1$ is evicted, the transaction cannot complete
- note: size of transaction is limited by cache size (in practice, smaller)
- upon abort, hardware needs to indicate whether to retry (synch. conflict) or not (error or SW-detected resource exhaustion)



HTM in the Intel Haswell/Broadwell

- has new instructions to begin, end and abort a transaction
 - `xbegin addr`: address of retry loop, in case of abort
 - `xend`: all loads/stores in between are transactional; and `xabort`
- no guarantee of progress! (indefinitely repeated aborts)
- e.g. `xbegin; load x; load y; store z;`

T	flag	tag
0	S	u
1	E	x
1	S	y
0	M	v
1	M	z
0	I	-

cache state after the above

T	flag	tag
0	S	u
0	I	x
0	I	y
0	M	v
0	I	z
0	I	-

if x's line becomes invalidated (remote core writes to x), before `xend`, abort

T	flag	tag
0	S	u
0	E	x
0	S	y
0	M	v
0	E	z
0	I	-

if v's line becomes invalidated (remote core writes to v), before `xend`, commit



Uses of HTM

- may be used to create custom atomic operations
- e.g. the **atomic double compare-and-swap** operation may be expressed as:

```

typedef unsigned long uint64;
2  int dcas(uint64 *x, uint64 x_old, uint64 x_new,
        uint64 *y, uint64 y_old, uint64 y_new) {
4      int swapped = 0;
      xbegin abortaddr;
      uint64 x_now = *x, y_now = *y;
      if (x_now == x_old && y_now == y_old)
8         *x = x_new, *y = y_new, swapped = 1;
      xend;
10     return (swapped); //success
      abortaddr:
12     return (0); //failure
}

```

- note: x86 provides a `cmpxchg16b` instruction which can do the above if x and y are in consecutive memory locations



Summary: Transactional Memory

- Atomic construct: aim to increase simplicity of synchronization without significantly sacrificing performance
- Implementation: many variants that differ in versioning policy (eager/lazy), conflict detection (pessimistic/optimistic), detection granularity
- Hardware transactional memory: versioned data kept in caches, conflict detection as part of coherence protocol